

Vehicle Maintenance Log

Introduction

This project is a small desktop application built in Java that helps a car owner keep track of three common maintenance tasks: engine oil changes, tyre rotations, and registration renewal. Instead of trying to remember dates and mileage numbers, the program keeps this information saved and tells the user when a task is due, based on how many miles have been driven or how many days have passed.

We built this project using Java Swing for the visual interface, and plain text file storage so that the car's information is remembered even after the program is closed and reopened.

Why We Made This

Cars need regular maintenance, and it is easy to forget when the last oil change happened or when the registration needs renewal. Most people either keep this in their head or write it on a sticky note, which does not always work well. This program solves that by keeping a small record of three important pieces of information: the last mileage the oil was changed, the last mileage the tyres were rotated, and the last date the registration was renewed. Using this, along with the car's current mileage, the program works out whether each task is due or not.

Files in the Project and What Each One Does

Main.java

This is the starting point of the program. It has the main method that Java looks for first when the program runs. All this file does is open the graphical window, CarMaintenanceGUI, on the screen. We used SwingUtilities.invokeLater so the window is created safely, which is the standard way to start any Swing application.

Info.java

This class is where the actual data lives. It stores four pieces of information: the current odometer reading, the mileage the oil was last changed, the mileage the tyres were last rotated, and the date the registration was last renewed.

Info.java also handles saving and loading this data to a file called CarFile.txt, using Java's built in Properties class. Every time something changes, such as the odometer being updated or a task being marked done, the class immediately saves the new values to the file. This way, if the user closes the program and opens it again later, none of the information is lost.

There is also a small safety check in this class. When the user tries to update the odometer, the setOdometer method makes sure the new reading is not smaller than the last recorded oil change mileage or tyre rotation mileage. This stops the user from accidentally entering a number that does not make sense, such as an odometer reading lower than a mileage that was already recorded.

Maintenance.java

This is an abstract class, which means it works as a blueprint rather than a class that gets used directly. It lists six methods that any maintenance class must have: three methods that return history text, like when a task was last done, and three methods that check whether something is due. We created this class to keep the design organized, so the Car class always knows exactly what jobs it is responsible for.

Car.java

This class extends Maintenance.java and fills in the six methods with real logic. It uses the data stored in Info.java to decide two things for each task: what to show as the history text, and whether the task is currently due.

For the engine oil, the rule we set is that oil should be changed every 5000 miles. The class checks the difference between the current odometer reading and the last mileage the oil was changed. If that difference is more than 5000, the program tells the user the oil change is due. Otherwise, it shows how many more miles are left before the next change.

The tyre rotation logic works the exact same way, except it uses a 1000 mile range instead of 5000.

Registration renewal works a bit differently since it depends on dates instead of mileage. The class checks how many days have passed since the last renewal date and compares that to a 365 day range. If no date has been recorded yet, it tells the user there is no registration date on record. If the stored date cannot be read properly for some reason, it shows a message saying the date is invalid, instead of the program crashing.

CarMaintenanceGUI.java

This is the biggest file in the project and it builds the entire visual window the user sees and interacts with. It is built using Java Swing components such as JFrame, JPanel, JLabel, and JButton.

The window is split into two main parts. At the top there is a header showing the app's title and a short description. Below that is the main content area, which holds an odometer section and three maintenance cards.

The odometer section shows the current mileage and a text box where the user can type in a new reading, along with an Update button next to it.

Each of the three maintenance cards, for engine oil, tyre rotation, and registration renewal, shows an icon, a short line of history text, a status line saying whether the task is due or how much mileage or time is left, and a Mark as Done button.

When the user presses Update after typing a new odometer number, the program checks whether the number is valid, meaning it must be a whole number and not lower than the mileages already recorded for oil or rotation. If everything checks out, the odometer is saved and every card on the screen refreshes to show the updated numbers. If the reading is not valid, a small warning window pops up explaining the problem, instead of the program failing silently.

When the user presses Mark as Done on the oil or tyre rotation card, the current odometer reading is saved as the new last service mileage for that task. For the registration card, pressing Mark as Done saves today's date as the last renewal date. In every case, the whole screen refreshes right after, so the user can immediately see the updated status.

The color of the status text also changes depending on whether a task is due. If a task is due, the text turns red. If it is not due yet, it turns green. This lets the user tell at a glance what needs attention without having to read every line closely.

ResetCounter.java

This is a very small class that is not yet connected to the rest of the program. It only has one boolean value and a reset method that returns it. It does not currently affect anything the user sees or does. We kept this file in the project because it may be used later to add a feature that resets all the maintenance records at once, but that feature has not been built yet.

How the Program Works

- The user opens the program, and Main.java starts the Swing application.
- CarMaintenanceGUI is created. While it is being set up, it also creates one Info object and one Car object. The Info object automatically tries to load any previously saved data from CarFile.txt. If the file does not exist yet, such as on the very first run, all the values simply start at zero or empty.
- The window builds all of its visual pieces: the header, the odometer section, and the three maintenance cards.

- Right after the window is built, it calls a refreshAll method, which pulls the latest data from Info and Car and displays it on all three cards and on the odometer label.
- From this point, the program simply waits for the user to interact with it.
- If the user updates the odometer, the new number is checked, saved into Info, which also writes it to the file, and the screen refreshes.
- If the user marks a task as done, Info is updated with the new mileage or date, saved to the file, and the screen refreshes again.
- This cycle of the user doing something, Info being updated and saved, and the screen refreshing, keeps repeating for as long as the program stays open.

Data Storage

All the important information the app needs to remember is kept in a plain text file called CarFile.txt, which sits in the same folder as the program. This file is written and read using Java's Properties class, which stores information as simple key and value pairs, one per line, such as odometer equals twelve thousand. Because the data is written to this file immediately after any change, the user never needs to press a separate save button anywhere in the app. Closing and reopening the program will always show the same numbers that were there before.

Conclusion

This project shows a simple but complete example of a Java desktop application. It brings together a graphical interface built with Swing, some basic logic to work out maintenance schedules based on mileage and dates, and file based storage so that information is never lost between runs. In the end, it gives a car owner an easy way to check whether an oil change, tyre rotation, or registration renewal is due, without needing to remember numbers and dates on their own.